

INDUSTRIAL INTERNSHIP REPORT

Data Science & MACHINE LEARNING INTERNSHIP

Submitted in partial fulfillment of the requirements for the award of the degree of

**Bachelor of Engineering in Computer Science and Engineering
(Artificial Intelligence and Machine Learning)**

Submitted by:

Md Fahad

Internship Organization:

Uniconverge Technologies(UCT)

Internship Partner:

Upskill Campus

In Association With

Edunet Foundation

Internship Duration:

July 2026 – August 2026

EXECUTIVE SUMMARY

Artificial Intelligence and Machine Learning have become important technologies for solving real-world problems across agriculture, transportation, healthcare, finance, education, and smart city management. The ability of Machine Learning systems to analyze historical data, identify patterns, and generate predictions provides organizations with valuable decision-support capabilities. As part of my Artificial Intelligence and Machine Learning internship with **Uniconverge Technologies in association with Edunet Foundation**, I worked on practical Machine Learning applications involving data preprocessing, exploratory analysis, model development, evaluation, deployment, and interactive web application development.

During the internship, I completed two major Machine Learning projects: **Agriculture Crop Production Prediction** and **Smart City Traffic Forecasting**. Both projects were developed with the objective of applying Machine Learning concepts to practical problems and transforming trained models into usable applications. The projects provided hands-on experience with the complete Machine Learning lifecycle, beginning with understanding the problem and dataset and continuing through preprocessing, feature engineering, model selection, training, evaluation, serialization, application development, and cloud deployment.

The first project, **Agriculture Crop Production Prediction**, focuses on estimating crop yield based on agricultural and economic parameters. Agriculture is highly dependent on several factors, including cultivation costs, production costs, geographical conditions, crop type, and regional characteristics. Predicting crop yield can assist farmers, agricultural planners, researchers, and policymakers in making better decisions. In this project, historical agricultural records were analyzed and used to train multiple regression algorithms. Linear Regression, Decision Tree Regression, and Random Forest Regression were evaluated using appropriate regression metrics. Among the tested models, Random Forest Regression achieved the best performance, with an R^2 score of approximately **94.63%**. The trained model was integrated into an interactive Streamlit application where users can select a crop

and state and provide cultivation and production cost information to obtain an estimated crop yield.

The second project, **Smart City Traffic Forecasting**, addresses the problem of predicting traffic volume at different city junctions. Rapid urbanization and increasing numbers of vehicles have created significant challenges for transportation management. Traffic congestion can result in increased travel time, fuel consumption, air pollution, and inefficient utilization of road infrastructure. A predictive system can help identify expected traffic conditions and support smarter transportation planning. In this project, a dataset containing **48,120 traffic records** was processed and transformed into suitable Machine Learning features. Three regression algorithms—Linear Regression, Decision Tree Regression, and Random Forest Regression—were trained and evaluated. The Random Forest model achieved the best performance with an R^2 score of **0.968963**, corresponding to approximately **96.90%**, with an MAE of **2.396650** and RMSE of **3.556561**.

Both projects were implemented using Python and commonly used Machine Learning and data analysis libraries, including NumPy, Pandas, and Scikit-learn. Joblib was used for model serialization, allowing the trained models to be stored and loaded by the deployed applications. Streamlit was used to develop interactive web interfaces that allow users without programming knowledge to interact with the Machine Learning models. The applications were designed with modern dashboard interfaces containing input controls, prediction cards, analytics sections, visualizations, and downloadable prediction reports.

The internship also provided practical experience in software development and deployment practices. In addition to developing the Machine Learning models, attention was given to application structure, dependency management, Git and GitHub version control, model file management, debugging, user interface development, and cloud deployment. The completed applications were deployed using Streamlit Community Cloud, making the Machine Learning solutions accessible through web browsers.

Overall, the internship provided an opportunity to move beyond theoretical knowledge and implement complete Machine Learning solutions. Working on two different domains—agriculture and smart transportation—helped develop a broader understanding of how Machine Learning can be adapted to different types of real-world problems. The projects also improved my understanding of regression algorithms, model evaluation, data preprocessing, application development, and deployment.

TABLE OF CONTENTS	Page.No.
1.Preface	6
2.Introduction	9
2.1. Background	
2.2. About UniConverge Technologies Pvt. Ltd	
2.3. About Upskill Campus	
2.4. Importance of Machine Learning	
2.5. Internship Objectives	
2.6. Scope of Internship	
2.7. Technologies Used	
2.8. Internship Methodology	
2.9. Report Organization	
3.Project - 1	
3.1. Introduction	18
3.2. Problem Statement	19
3.3. Motivation	20
3.4. Existing system Vs Proposed system	21
3.5. Objectives of the project	25
3.6. Dataset Description	26
3.7. Feature description	27
3.8. Data preprocessing	28
3.9. Machine Learning Algorithms	30
3.10. Model Evaluation	33
3.11. Streamlit application	35
3.12. Project 1 Output	38
4.Project - 2	
4.1. Introduction	41
4.2. Problem Statement	42
4.3. Motivation	43
4.4. Existing System vs Proposed System	44
4.5. Objectives of the project	47
4.6. Dataset description	48
4.7. Feature description	49
4.8. Data Preprocessing	51
4.9. Machine Learning Algorithms	54

4.10. Model Evaluation	57
4.11. Streamlit Application	61
4.12. Results & Discussion	63
5. Performance Test for Project 1	64
5.1. Test Cases	64
5.2. Test Procedure	65
5.3. Performance Outcome	65
6. Performance Test for Project 2	66
6.1. Test Cases	66
6.2. Test Procedure	67
6.3. Performance Outcome	67
7. My Learnings	68
8. Future Work Scope	69
9. References	70

1. PREFACE

The Industrial Internship was undertaken as an important component of the Bachelor of Engineering program in Computer Science and Engineering with a specialization in Artificial Intelligence and Machine Learning. The primary purpose of the internship was to gain practical exposure to technologies and methodologies used in the development of real-world software and Machine Learning applications.

Academic coursework provides a strong foundation in programming, algorithms, databases, Machine Learning, and software engineering. However, implementing these concepts in real projects requires additional practical knowledge. The internship provided an opportunity to understand how theoretical concepts are transformed into functional applications. It also helped in developing problem-solving abilities, debugging skills, documentation practices, and an understanding of the complete software development lifecycle.

During the internship, I worked on two Machine Learning projects focused on different real-world domains. The first project involved predicting agricultural crop production, while the second project focused on forecasting traffic volume in a smart city environment. Although both applications use Machine Learning regression techniques, the nature of their datasets, features, objectives, and application requirements differs significantly.

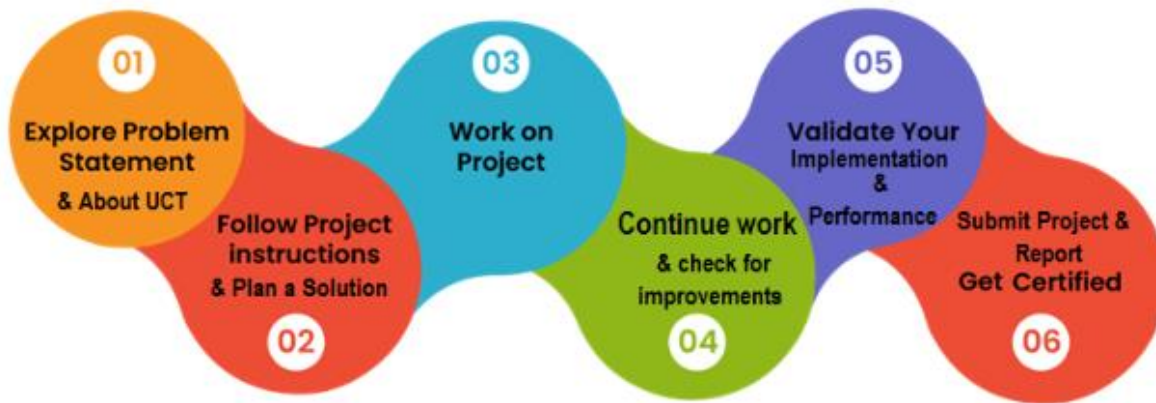
The Agriculture Crop Production Prediction project provided an opportunity to understand how economic and agricultural parameters can be used to estimate crop yield. The project involved categorical feature encoding, numerical feature processing, model training, model comparison, and deployment through Streamlit. The project demonstrated how Machine Learning can be used to support agricultural decision-making.

The Smart City Traffic Forecasting project provided experience in processing a comparatively larger dataset containing tens of thousands of traffic observations. The project involved date and time feature engineering, traffic volume prediction, regression model comparison, visualization, and development of an interactive forecasting dashboard. The application was designed to provide users with predicted traffic volume and an indication of traffic intensity.

An important aspect of the internship was learning how to convert Machine Learning models into usable software applications. Instead of limiting the work to Jupyter Notebook experiments, the trained models were integrated into Streamlit applications. This required understanding model serialization, project directory structures, dependency management, user input handling, error handling, interface design, and deployment.

The internship also provided experience with GitHub for source code management and Streamlit Community Cloud for application deployment. During development, several technical challenges were encountered, including model loading issues, dependency compatibility, user input encoding, prediction state management, large Machine Learning model files, and deployment configuration. Solving these issues improved practical debugging and troubleshooting abilities.

This report presents the complete work performed during the internship. It explains the background of the internship, problem statements, datasets, methodologies, Machine Learning algorithms, implementation procedures, application interfaces, model evaluation, testing, deployment, challenges, learning outcomes, and future improvements associated with both projects.



I would like to express my sincere gratitude to the mentors and organizers of Upskill Campus and UniConverge Technologies Pvt. Ltd. for providing valuable guidance and an excellent learning environment throughout the internship duration.

2. INTRODUCTION

2.1 Background

Artificial Intelligence is one of the most rapidly developing areas of computer science. It enables computer systems to perform tasks that traditionally require human intelligence, including pattern recognition, prediction, classification, decision-making, and natural language understanding. Machine Learning is a major branch of Artificial Intelligence that enables systems to learn patterns from historical data rather than relying entirely on manually programmed rules.

Machine Learning has become particularly useful in situations where large amounts of historical data are available. By identifying relationships between input variables and desired outcomes, Machine Learning models can generate predictions for previously unseen data. Regression algorithms are especially useful when the target variable is numerical, such as crop yield, sales volume, temperature, energy consumption, or traffic volume.

The practical value of Machine Learning depends not only on model accuracy but also on how effectively the model can be integrated into an application. A highly accurate model that cannot be easily accessed by users has limited practical value. Therefore, modern Machine Learning projects increasingly involve the complete pipeline from data collection and preprocessing to model deployment and user interaction.

The internship provided practical exposure to this complete pipeline. Instead of focusing exclusively on algorithmic concepts, the projects required understanding the entire process of developing and deploying Machine Learning applications.

2.2 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end etc.



UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

* It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

* It supports both cloud and on-premises deployments.

It has features to
• Build Your own dashboard

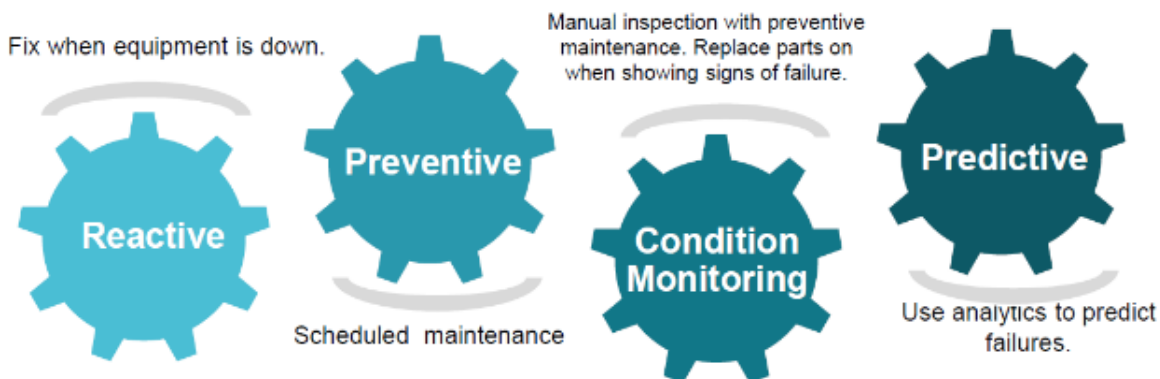
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine

2.3 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers personalized executive coaching in a more affordable, scalable and measurable way.

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.



2.4 Importance of Machine Learning in Real-World Applications

Traditional software systems generally operate according to explicitly defined rules. For example, a conventional program may calculate a value using a predefined mathematical formula. Such approaches work well when the underlying relationship between input and output is known and relatively stable.

However, many real-world problems are more complex. The relationship between different variables may not be easily represented through manually written rules. Machine Learning provides an alternative approach by allowing models to learn relationships from historical observations.

In agriculture, for example, crop production can be influenced by crop type, location, cultivation expenditure, production expenditure, weather conditions, soil properties, and farming practices. Creating a fixed mathematical rule that accurately captures all these relationships can be difficult. A Machine Learning model can instead learn patterns from historical agricultural data and use those patterns to estimate future outcomes.

Similarly, traffic conditions are affected by junction location, time of day, day of week, seasonal patterns, and historical traffic behavior. Traffic forecasting is therefore another suitable application for Machine Learning.

These examples demonstrate the ability of Machine Learning to support decision-making in domains where historical data contains useful patterns.

2.5 Internship Objectives

The major objective of the internship was to gain practical experience in Artificial Intelligence and Machine Learning by developing functional applications based on real-world datasets.

The specific objectives included developing an understanding of the complete Machine Learning workflow, including dataset analysis, preprocessing, feature engineering, model selection, model training, evaluation, and deployment.

Another important objective was to compare different Machine Learning algorithms and identify the model that provides the best performance for a particular problem. Instead of assuming that one algorithm is always superior, multiple regression algorithms were implemented and evaluated using appropriate performance metrics.

The internship also aimed to develop practical Python programming skills and gain experience with important libraries such as Pandas, NumPy, Scikit-learn, Joblib, and Streamlit.

An additional objective was to learn how Machine Learning models can be transformed into interactive applications. The Streamlit framework was used to create user-friendly interfaces through which users can provide input values and receive predictions without directly interacting with Python code.

Finally, the internship aimed to develop software engineering practices including project organization, GitHub version control, dependency management, debugging, documentation, and cloud deployment.

2.6 Scope of the Internship

The scope of the internship covered the development of two independent Machine Learning applications.

The first application focused on agriculture and aimed to predict crop yield from agricultural cost-related parameters and categorical information such as crop and state. The application demonstrates the potential use of predictive analytics in agriculture.

The second application focused on smart city transportation and aimed to predict traffic volume at different junctions based on date and time-related features. The application demonstrates how historical traffic information can be used to support transportation analysis.

The scope also included development of interactive dashboards, visualization of prediction results, model serialization, deployment, and documentation.

The projects were developed primarily for educational and demonstration purposes. Although the models achieved strong performance on their respective test datasets, real-world production systems would require additional data, continuous validation, real-time data sources, and domain-specific evaluation before being used for operational decision-making.

2.7 Technologies Used

The projects were developed using Python as the primary programming language. Python was selected because of its extensive ecosystem for data science and Machine Learning.

NumPy was used for numerical operations and preparation of model input arrays. **Pandas** was used for data loading, preprocessing, manipulation, and analysis. **Scikit-learn** provided the Machine Learning algorithms and evaluation metrics used during model development.

Joblib was used to serialize trained Machine Learning models and encoders. This allowed the models to be saved after training and loaded later by the Streamlit applications without retraining them every time the application starts.

Streamlit was used to transform the trained models into interactive web applications. It provided input widgets such as dropdown menus, sliders, date selectors, buttons, metrics, tables, and charts.

Git and GitHub were used for source code management and project version control. The applications were eventually deployed using **Streamlit Community Cloud**, allowing them to be accessed through public web URLs.

2.8 Internship Methodology

The development methodology followed during the internship can be represented as a sequence of interconnected stages.

First, the problem statement was studied to understand the objective of the application and determine what type of prediction was required. The available dataset was then inspected to understand its structure, attributes, data types, and target variable.

The next stage involved data preprocessing. Duplicate records, missing values, inconsistent formats, and unsuitable attributes were examined. Features were transformed into formats appropriate for Machine Learning algorithms.

After preprocessing, the datasets were divided into training and testing portions. Multiple regression algorithms were then trained using the training data.

The trained models were evaluated using metrics such as Mean Absolute Error, Root Mean Squared Error, and R^2 Score. The model demonstrating the strongest performance was selected for the final application.

The selected model was serialized using Joblib. A Streamlit application was then developed to provide a user-friendly interface for interacting with the model.

Finally, the application was tested locally, debugging issues were resolved, and the completed applications were deployed using Streamlit Community Cloud.

The overall methodology can therefore be summarized as:

Problem Understanding → Data Collection → Data Analysis → Preprocessing → Feature Engineering → Train/Test Split → Model Training → Model

**Evaluation → Model Selection → Model Serialization → Streamlit
Development → Testing → Deployment**

2.9 Report Organization

This report is organized into multiple chapters to provide a detailed explanation of the internship activities and both Machine Learning projects.

The introductory chapters discuss the internship background, objectives, technologies, and methodology. The subsequent chapters explain the Agriculture Crop Production Prediction project in detail, including its problem statement, dataset, preprocessing methodology, algorithms, implementation, user interface, testing, and results.

The report then presents the Smart City Traffic Forecasting project using a similar structure. This includes detailed discussion of the traffic dataset, feature engineering, model development, performance evaluation, dashboard implementation, visualization, and deployment.

The later chapters combine the findings from both projects and discuss challenges encountered during development, skills acquired during the internship, limitations of the developed systems, possible future improvements, and the overall conclusion of the internship.

3. PROJECT 4 — AGRICULTURE CROP PRODUCTION PREDICTION

3.1 Introduction

Agriculture plays an important role in the economic and social development of a country. Crop production is influenced by several factors, including the type of crop being cultivated, geographical location, cultivation expenditure, production expenditure, availability of resources, and farming conditions. Accurate estimation of crop yield can therefore provide useful information for farmers, agricultural planners, researchers, and other stakeholders.

Traditional crop production estimation methods often depend heavily on historical averages, manual calculations, expert assumptions, and field-level observations. Although these methods can provide useful information, they may not be able to effectively capture complex relationships between multiple variables. Machine Learning provides an alternative approach by learning patterns from historical agricultural data and using those patterns to generate predictions for new input conditions.

The **Agriculture Crop Production Prediction** project was developed to demonstrate the application of supervised Machine Learning regression techniques to crop yield prediction. The system accepts information related to the crop, state, cultivation costs, and production cost and generates an estimated crop yield in **Quintal per Hectare**.

The project was developed as an end-to-end Machine Learning application. The work began with understanding and preprocessing the agricultural dataset, followed by feature encoding, model training, model comparison, and evaluation. The best-performing model was then saved using Joblib and integrated into a Streamlit web application.

The final application provides a simple interface through which users can select a crop and state, enter agricultural cost information, and receive a predicted crop yield.

The application also provides a summary of the entered information and displays the predicted yield in a visually clear result card.

3.2 Problem Statement

Crop yield prediction is a challenging problem because agricultural production depends on multiple variables. Different crops can have different production characteristics, while the same crop can produce different yields depending on the state or geographical region in which it is cultivated.

The objective of this project is to develop a Machine Learning-based system capable of predicting expected crop yield using available historical agricultural information.

The system should accept relevant agricultural parameters as input and use a trained regression model to estimate the expected crop yield. The predicted value should be presented in a user-friendly web interface so that users can easily understand the output.

The project therefore addresses the following problem:

To develop and deploy a Machine Learning regression system that predicts crop yield based on crop type, state, cultivation cost, and production cost.

The project focuses on demonstrating the practical application of Machine Learning rather than replacing professional agricultural decision-making. The prediction generated by the system should therefore be considered an analytical estimate based on the patterns learned from the available dataset.

3.3 Motivation

The motivation for developing this project comes from the increasing importance of data-driven decision-making in agriculture. Agricultural planning requires understanding historical production patterns and identifying factors that may influence crop performance.

A predictive system can help demonstrate how historical agricultural records can be transformed into useful information. Instead of manually analyzing individual records, a Machine Learning model can learn relationships between multiple variables and produce a prediction automatically.

Another motivation was to gain practical experience in applying regression algorithms to a real-world dataset. The project provided an opportunity to compare different algorithms rather than relying on a single model.

The project also aimed to demonstrate the complete transition from a Machine Learning experiment to a deployable application. The trained model was integrated into Streamlit so that users could interact with the system without writing Python code.

3.4 Existing Approach

Traditional approaches to crop yield estimation generally rely on historical records, statistical calculations, agricultural surveys, field observations, and expert knowledge. Historical production values can be analyzed to estimate expected yield based on previous trends.

Such approaches can be useful when the agricultural environment remains relatively stable. However, manually analyzing multiple parameters becomes difficult when the number of variables increases.

For example, crop type and state may represent categorical differences, while cultivation and production costs represent numerical information. An analytical system needs to consider these variables together to generate a prediction.

Machine Learning provides a way to automatically identify relationships between these variables based on historical observations.

3.5 Limitations of Traditional Approaches

One limitation of manual prediction methods is that they may require considerable human effort. When multiple parameters need to be considered simultaneously, manual analysis can become time-consuming.

Another limitation is that traditional methods may not easily capture non-linear relationships between variables. Agricultural datasets can contain complex patterns that are difficult to represent using simple formulas.

Traditional approaches can also make it difficult to provide instant predictions for different combinations of input values. A computerized Machine Learning system can perform this process much more efficiently after the model has been trained.

Therefore, the project proposes the use of a supervised Machine Learning regression model to automate the prediction process.

3.6 Proposed System

The proposed system uses historical agricultural data to train regression models capable of predicting crop yield.

The system receives five major input features:

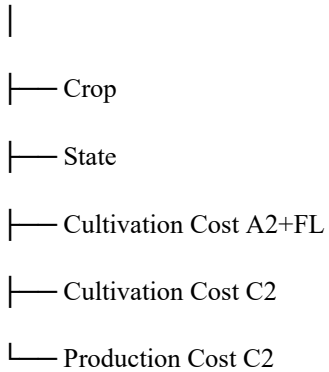
1. Crop
2. State
3. Cultivation Cost $A2+FL$
4. Cultivation Cost $C2$
5. Production Cost $C2$

The categorical variables **Crop** and **State** are converted into numerical representations using trained encoders. The numerical agricultural cost values are then combined with the encoded categorical features.

The resulting feature vector is passed to the trained Machine Learning model. The model generates the predicted crop yield, which is displayed to the user through the Streamlit dashboard.

The overall process can be represented as:

User Input



Categorical Encoding



Feature Preparation



Random Forest Model



Yield Prediction



Streamlit Result Dashboard

3.7 Objectives of the Project

The primary objective of the project is to develop a predictive Machine Learning system for estimating agricultural crop yield.

The project has several supporting objectives. The first is to understand the structure of the agricultural dataset and identify useful features for prediction. The second is to preprocess categorical and numerical data so that it can be provided to Machine Learning algorithms.

Another objective is to train multiple regression models and compare their performance using suitable evaluation metrics. This allows the best-performing model to be selected based on actual experimental results.

The project also aims to create a practical web interface using Streamlit. The interface should make the prediction process simple enough that users can provide inputs without interacting directly with the underlying Machine Learning implementation.

Finally, the project aims to deploy the completed application so that the prediction system can be accessed through a web browser.

3.8 Dataset Description

The Agriculture Crop Production Prediction project uses a historical agricultural dataset containing information related to crop production and associated costs.

The dataset contains **49 records** and includes categorical as well as numerical attributes.

The available attributes are:

Feature	Type	Description
Crop	Categorical	Name/type of crop
State	Categorical	State associated with the agricultural record
Cost of Cultivation A2+FL	Numerical	Cultivation cost per hectare under A2+FL
Cost of Cultivation C2	Numerical	Cultivation cost per hectare under C2
Cost of Production C2	Numerical	Production cost per quintal under C2
Yield	Numerical	Crop yield in Quintal/Hectare

The **Yield** attribute is the target variable because the purpose of the project is to predict crop yield.

3.9 Feature Description

Crop

The Crop feature identifies the agricultural crop associated with each observation. Since crop names are categorical values, they cannot be directly passed to standard Machine Learning regression algorithms in their textual form.

Therefore, the crop values were transformed into numerical representations using a trained encoder.

State

The State feature represents the geographical state associated with the agricultural record. Different states can have different agricultural characteristics. Therefore, this information was retained as a predictive feature.

Similar to Crop, State is a categorical variable and was encoded before being passed to the model.

Cost of Cultivation A2+FL

This feature represents the cost of cultivation per hectare under the A2+FL category. It provides information about agricultural expenditure associated with cultivation and is used as one of the numerical predictors.

Cost of Cultivation C2

The Cost of Cultivation C2 feature represents another measure of cultivation expenditure per hectare. This feature provides additional information regarding the economic characteristics associated with producing a crop.

Cost of Production C2

The Cost of Production C2 feature represents production cost per quintal. This variable provides information about the cost associated with producing the agricultural output and is included as an input to the regression model.

Yield

Yield is the target variable of the project. The objective of the Machine Learning model is to estimate this value based on the other available features.

3.10 Data Preprocessing

Data preprocessing is an important stage of the Machine Learning workflow because the quality and representation of input data directly affect the ability of a model to learn useful patterns.

The agricultural dataset contains both categorical and numerical features. Categorical variables such as Crop and State cannot be directly processed by the regression models in their original textual format. Therefore, these variables were transformed into numerical representations.

The numerical cost-related features were retained as numerical values and supplied to the model after appropriate validation. The preprocessing stage also involved checking the dataset structure and preparing the data for model training and testing.

Categorical Feature Encoding

Two categorical features were used in the project:

- Crop
- State

A label encoding approach was used to convert the categorical values into numerical values.

For example, conceptually, a categorical list such as:

Crop A
Crop B
Crop C

can be represented internally as numerical values such as:

0
1
2

The exact encoded values depend on the classes learned by the encoder during training.

The trained encoders were saved separately using Joblib:

model/

|

|— crop_prediction_model.pkl

|— crop_encoder.pkl

|— state_encoder.pkl

This is important because the same encoding mapping used during model training must also be used when a user provides new input through the application.

Train-Test Split

The available agricultural dataset was divided into training and testing portions.

The training data was used to allow the Machine Learning algorithms to learn relationships between the input variables and crop yield.

The testing data was kept separate and used to evaluate how well the trained models performed on unseen observations.

The project used:

Dataset	Shape
Training Data	39×5
Testing Data	10×5

The target variable was excluded from the input feature matrix during prediction and used as the expected output during model training and evaluation.

3.11 Machine Learning Algorithms

Three regression algorithms were evaluated during the development of the project:

1. Linear Regression
2. Decision Tree Regression
3. Random Forest Regression

The purpose of testing multiple algorithms was to identify the model that provided the strongest predictive performance on the available dataset.

Linear Regression

Linear Regression is a supervised Machine Learning algorithm used for predicting continuous numerical values.

The algorithm attempts to establish a linear relationship between the input variables and the target variable.

In the context of this project, Linear Regression was used as a baseline model. The model attempts to estimate crop yield as a function of the available agricultural features.

The advantage of Linear Regression is its simplicity and interpretability. However, its performance may be limited when the relationship between the features and target variable is non-linear.

The model achieved:

MAE: 116.76

RMSE: 140.98

R² Score: 0.78

Although the model demonstrated some predictive capability, another algorithm performed better on the available dataset.

Decision Tree Regression

Decision Tree Regression is a non-linear supervised Machine Learning algorithm.

Instead of fitting a single mathematical equation, the algorithm creates a tree-like structure that divides the feature space into different regions. Each decision is based on feature values, and the final leaf nodes represent predicted numerical values.

Decision Trees can model non-linear relationships and interactions between features.

In this project, Decision Tree Regression achieved:

MAE: 60.94

RMSE: 179.24

R² Score: 0.6444

Although the Decision Tree improved some aspects compared with the baseline model, it did not achieve the overall performance of the Random Forest model.

Random Forest Regression

Random Forest Regression is an ensemble Machine Learning algorithm that combines multiple decision trees to produce a final prediction.

Instead of depending on a single decision tree, Random Forest constructs multiple trees and combines their predictions. This generally improves robustness and reduces the risk of relying too heavily on the patterns learned by a single tree.

For the Agriculture Crop Production Prediction project, Random Forest produced the strongest results among the tested models.

The model achieved:

MAE: 28.43

RMSE: 69.66

R² Score: 0.9463

The R² score of approximately **94.63%** indicated that Random Forest was the best-performing model among the evaluated algorithms for this project.

Therefore, Random Forest Regression was selected as the final prediction model.

Model Comparison

The experimental results are summarized below.

Model	MAE	RMSE	R ² Score
Linear Regression	116.76	140.98	0.78
Decision Tree	60.94	179.24	0.6444
Random Forest	28.43	69.66	0.9463

The comparison clearly shows that Random Forest achieved the highest R² score and the lowest MAE among the evaluated models.

Based on these results, Random Forest was selected for integration into the final Streamlit application.

3.12 Model Evaluation

Three evaluation metrics were considered during model comparison: Mean Absolute Error, Root Mean Squared Error, and R^2 Score.

Mean Absolute Error (MAE) represents the average absolute difference between actual and predicted values. Lower MAE values indicate that predictions are, on average, closer to the actual values.

Root Mean Squared Error (RMSE) measures the square root of the average squared prediction error. Because the errors are squared before averaging, larger errors have greater influence on this metric.

R^2 Score measures how well the model explains variation in the target variable. A higher R^2 score indicates stronger explanatory and predictive performance on the evaluation data.

The Random Forest model provided the strongest overall combination of these metrics.

Why Random Forest Was Selected

Random Forest was selected because it provided the best performance among the three evaluated algorithms.

The model achieved an R^2 score of **0.9463**, compared with 0.78 for Linear Regression and 0.6444 for Decision Tree Regression.

The Random Forest model also produced the lowest MAE among the tested models. This indicates that the model's predictions were, on average, closer to the actual yield values.

Another advantage of Random Forest is its ability to model non-linear relationships. Agricultural variables may interact in ways that cannot easily be represented by a simple linear equation.

For these reasons, Random Forest was considered the most suitable model for the final application.

Model Serialization

After selecting the Random Forest model, it was serialized using the Joblib library.

The purpose of serialization is to save the trained Machine Learning model to a file so that it can be loaded later without repeating the training process.

The primary model file is:

`crop_prediction_model.pkl`

The categorical encoders were also saved:

`crop_encoder.pkl`

`state_encoder.pkl`

The final model directory therefore contains:

```
model/
├──
├── crop_prediction_model.pkl
├── crop_encoder.pkl
└── state_encoder.pkl
```

During application startup, these files are loaded using Joblib.

3.13 Streamlit Application

After completing the Machine Learning model, the next stage was to convert the model into an interactive web application.

Streamlit was selected because it provides a simple Python-based framework for creating data science and Machine Learning web applications.

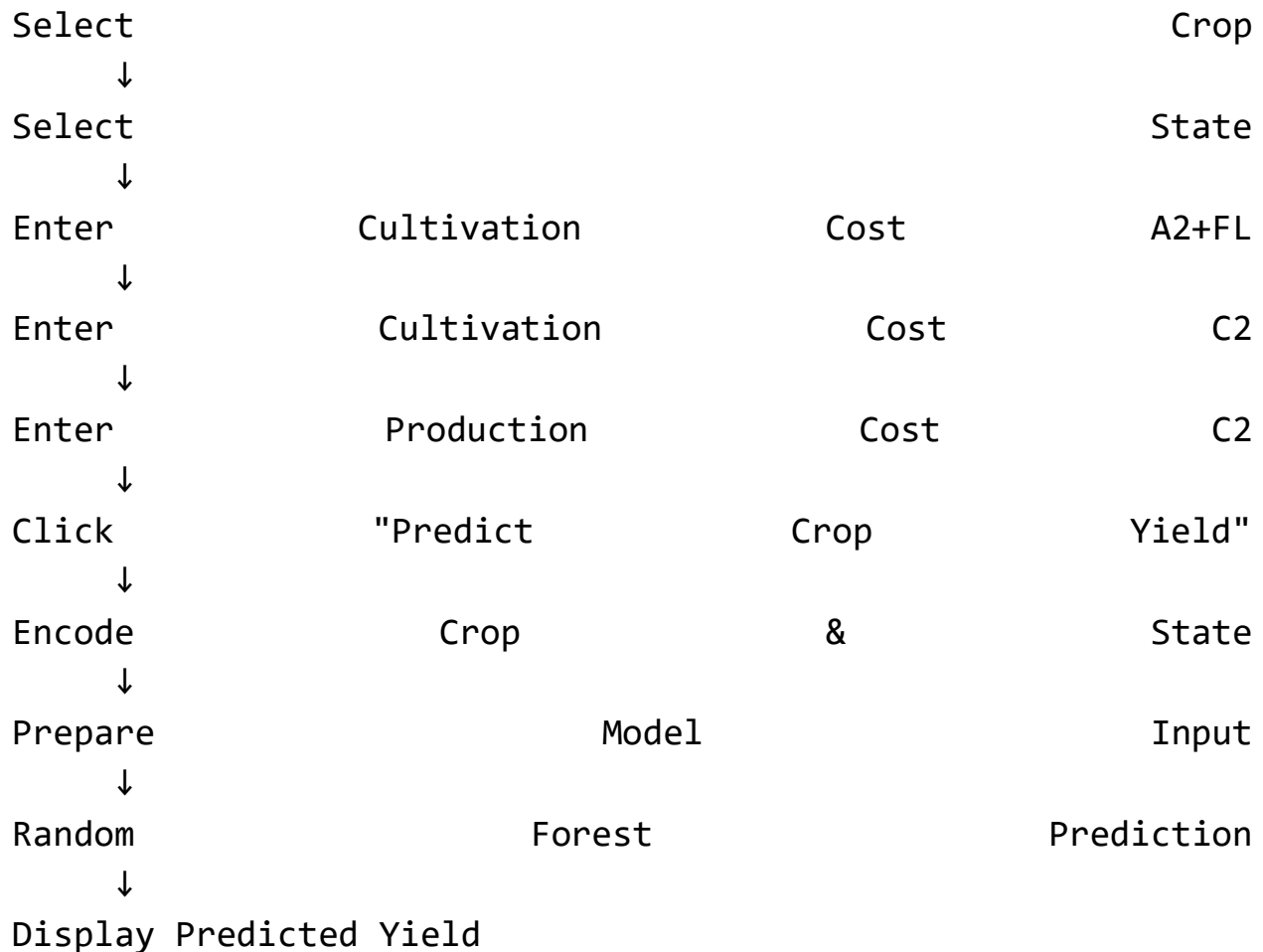
The application provides users with dropdown menus for selecting crop and state and numerical input fields for entering agricultural cost values.

The application then performs the same preprocessing used during training and passes the resulting feature vector to the saved Random Forest model.

The prediction is displayed in a visually highlighted result section.

User Input Workflow

The application follows the following workflow:



This workflow makes the application straightforward for users who do not have direct knowledge of Machine Learning programming.

User Interface Design

The Streamlit application was designed with a modern dashboard-oriented interface.

The interface contains a title section introducing the Agriculture Crop Production Prediction system. A short description explains the purpose of the application.

The dashboard also contains information cards displaying the selected Machine Learning model, model performance, and dataset size.

The input section provides clearly labeled controls for selecting crop and state and entering numerical agricultural cost values.

Once the user clicks the prediction button, the application generates the result and displays it inside a highlighted prediction card.

The interface was intentionally designed to separate input information from prediction results so that users can easily understand the relationship between the values they entered and the generated output.

Prediction Process

When the user clicks the **Predict Crop Yield** button, the application first converts the selected Crop and State values into their corresponding encoded numerical representations.

The encoded categorical features are then combined with the numerical cultivation and production cost values.

The resulting array is structured according to the same feature order used during model training.

The Random Forest model receives this array and generates the predicted yield.

The predicted value is then formatted to two decimal places and displayed as:

Quintal / Hectare

This ensures that the prediction is presented in a meaningful agricultural unit.

3.14 Project Output

The final application produces an estimated crop yield based on the user's selected agricultural inputs.

For example, after entering a valid combination of crop, state, and cost information, the application may generate a result such as:

Predicted	Crop	Yield
-----------	------	-------

16.23 Quintal / Hectare

The actual output varies depending on the input values supplied by the user.

The system therefore provides dynamic predictions rather than displaying a fixed result.

Project 4 Results

The Agriculture Crop Production Prediction project successfully demonstrated the application of Machine Learning regression to agricultural yield prediction.

Three algorithms were tested, and Random Forest Regression achieved the strongest performance with an R^2 score of **0.9463**.

The final trained model was successfully integrated into a Streamlit web application. The application allows users to provide agricultural information and obtain a prediction through an interactive interface.

The project therefore successfully completed the complete pipeline from dataset processing to Machine Learning model deployment.

Figure 2.1 — Agriculture Project Home Dashboard

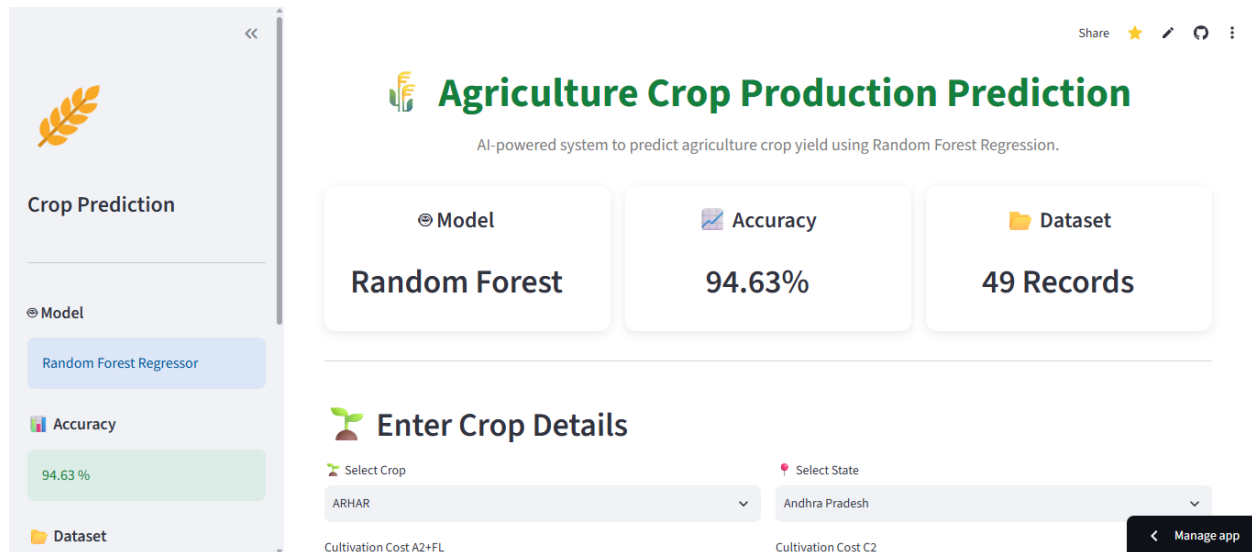


Figure 2.2 — Crop and State Selection

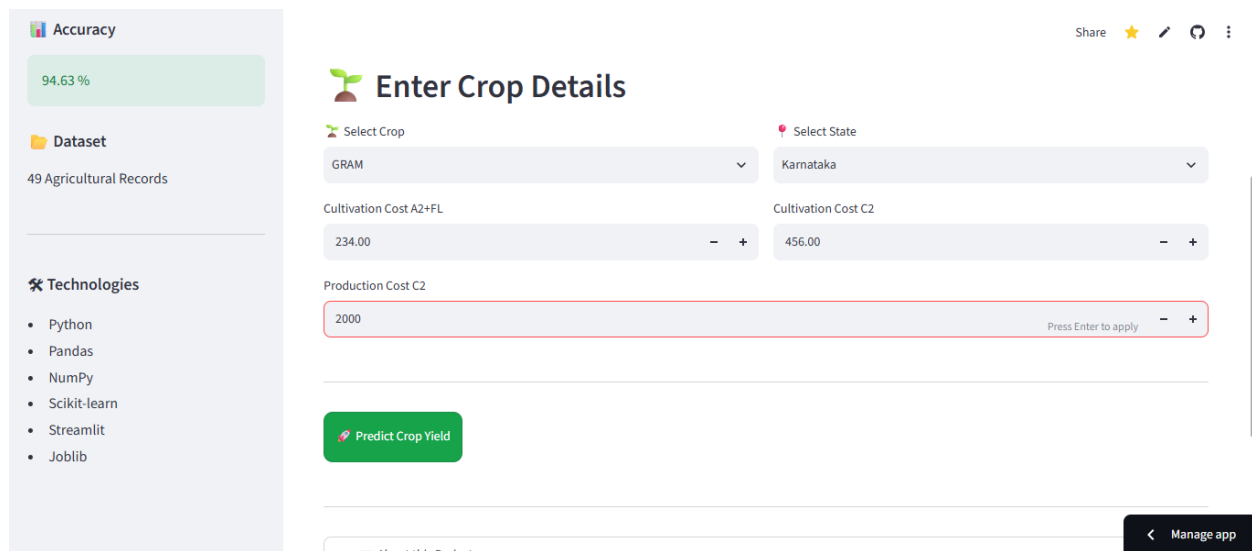


Figure 2.3 — Prediction Result

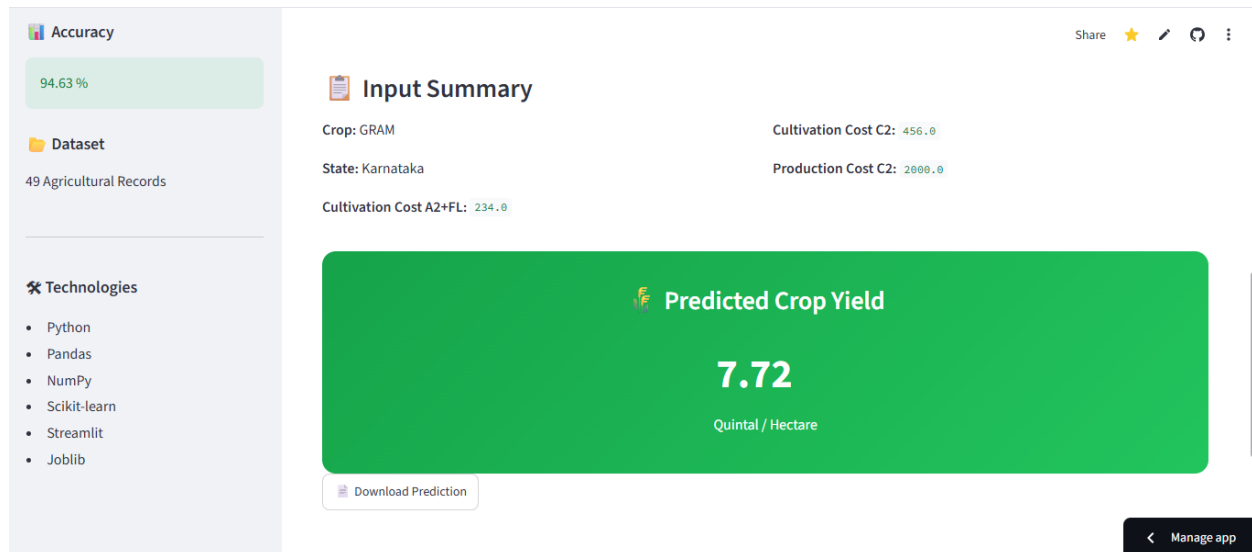
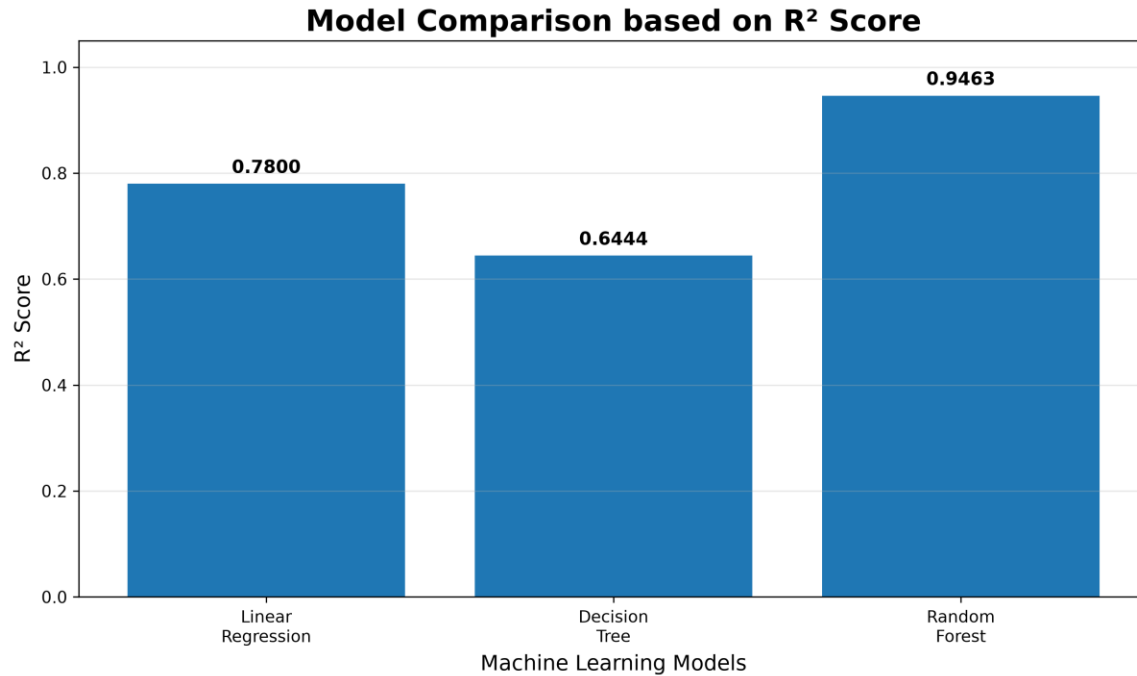


Figure 2.4 — Model Performance Comparison



4. PROJECT 9 — SMART CITY TRAFFIC FORECASTING

4.1 Introduction

The rapid growth of urban populations has significantly increased the number of vehicles on roads, leading to severe traffic congestion in cities around the world. Traffic congestion affects not only travel time but also fuel consumption, environmental pollution, public safety, and the overall quality of life of citizens. As governments move toward the concept of Smart Cities, intelligent traffic management has become one of the most important areas where Artificial Intelligence and Machine Learning can provide practical solutions.

A Smart City utilizes digital technologies, data analytics, sensors, Internet of Things (IoT) devices, and Artificial Intelligence to improve urban infrastructure and enhance public services. Among these services, traffic management plays a critical role because transportation directly influences economic productivity, emergency response systems, and daily commuting experiences. Predicting future traffic conditions allows city planners and traffic authorities to make informed decisions regarding road infrastructure, traffic signal optimization, and congestion management.

Traditional traffic monitoring systems mainly depend on historical observations and manual analysis. Although these methods provide useful information, they often struggle to capture complex traffic patterns that change according to time, day of the week, location, and seasonal trends. Machine Learning techniques overcome these limitations by learning hidden relationships from historical traffic datasets and generating accurate predictions for future traffic volumes.

The **Smart City Traffic Forecasting** project was developed as part of the Machine Learning Internship to demonstrate the application of supervised regression algorithms for predicting traffic volume at different city junctions. The project uses historical traffic data collected from four junctions over multiple years. Various machine learning algorithms were trained and compared to determine the most

suitable prediction model. The best-performing model was integrated into a professional Streamlit dashboard, allowing users to forecast traffic by selecting the junction, date, and hour.

The completed application provides an intuitive user interface with prediction cards, traffic status indicators, trend visualization, downloadable reports, and an interactive dashboard. This project demonstrates the complete Machine Learning lifecycle, including data preprocessing, feature engineering, model development, evaluation, deployment, and user interaction.

4.2 Problem Statement

Modern cities experience continuously changing traffic patterns due to population growth, increasing vehicle ownership, festivals, working days, weekends, holidays, weather conditions, and special events. Managing traffic efficiently requires accurate forecasting of future traffic conditions rather than reacting after congestion has already occurred.

The Government aims to transform conventional cities into Smart Cities capable of making intelligent transportation decisions using data-driven approaches. Traffic authorities require predictive systems that can estimate traffic volume in advance so that congestion can be minimized through proactive planning.

The challenge is to build a Machine Learning system capable of accurately forecasting the number of vehicles expected at different city junctions based on historical traffic information.

The developed system should:

- Accept traffic-related input parameters from the user.
- Analyze historical traffic patterns learned during training.
- Predict expected vehicle count.
- Classify traffic conditions into Low, Moderate, or Heavy traffic.
- Present the prediction through an interactive web dashboard.

The project therefore addresses the following problem:

To develop and deploy a Machine Learning-based Smart City Traffic Forecasting System capable of predicting future traffic volume using historical traffic data and temporal features.

4.3 Motivation

Traffic congestion has become one of the major challenges faced by rapidly growing cities. Long waiting times at intersections increase fuel consumption, reduce productivity, contribute to air pollution, and create inconvenience for commuters. Intelligent traffic forecasting can help authorities identify congestion before it occurs and implement preventive measures.

The motivation behind this project was to understand how Machine Learning can contribute to intelligent transportation systems. Rather than relying solely on historical averages, Machine Learning models can identify hidden patterns and relationships between multiple variables such as time, date, and junction location.

Another motivation was to gain practical experience in developing an end-to-end predictive analytics solution. The project allowed the application of regression algorithms on a large real-world dataset containing more than **48,000 traffic records**. It also provided valuable experience in feature engineering, model evaluation, deployment using Streamlit, and building a professional dashboard.

The project also aimed to demonstrate how Machine Learning models can be transformed into real-world applications that provide useful predictions through a simple graphical interface. Such systems have potential applications in intelligent traffic signal control, urban planning, route optimization, emergency vehicle management, and smart transportation infrastructure.

4.4 Existing System

Conventional traffic management systems generally rely on traffic police observations, fixed traffic schedules, CCTV monitoring, historical statistics, and manually analyzed reports. While these methods provide basic monitoring capabilities, they often fail to predict future traffic conditions accurately.

Many traditional systems operate reactively rather than proactively. Traffic congestion is usually managed after it has already occurred instead of anticipating future traffic peaks. Such approaches can result in inefficient traffic signal timings, delayed emergency response, and increased travel times.

Earlier statistical forecasting techniques primarily relied on averages and simple mathematical models. Although these methods work reasonably well under stable conditions, they struggle to capture nonlinear relationships present in traffic data.

Moreover, traffic volume changes continuously throughout the day. Morning office hours, evening rush hours, weekends, public holidays, and seasonal variations create complex patterns that are difficult to model using conventional analytical techniques.

These limitations highlight the need for intelligent predictive systems capable of learning traffic behavior from historical observations.

4.5 Limitations of Existing System

The traditional traffic forecasting approach suffers from several limitations.

One major limitation is the dependence on manual monitoring and analysis. Human observation is time-consuming and cannot efficiently process large volumes of continuously generated traffic data.

Another limitation is the inability of traditional mathematical models to capture nonlinear relationships between different traffic variables. Traffic behavior depends on multiple interacting factors, making prediction challenging using fixed statistical formulas.

Conventional systems also lack real-time prediction capability. Instead of forecasting future traffic conditions, they often provide information only after congestion has already occurred.

Furthermore, many existing approaches cannot automatically adapt to changing traffic patterns as cities expand and transportation networks evolve.

Finally, manual traffic analysis cannot easily scale to large smart city environments containing multiple intersections and continuously changing traffic conditions.

4.6 Proposed System

The proposed Smart City Traffic Forecasting System utilizes Machine Learning techniques to predict traffic volume based on historical traffic data. The system accepts several user inputs, including junction number, selected date, hour of the day, and day of the week.

These input features are converted into the numerical format required by the trained Machine Learning model. The processed feature vector is then passed to the Random Forest Regression model, which predicts the expected number of vehicles.

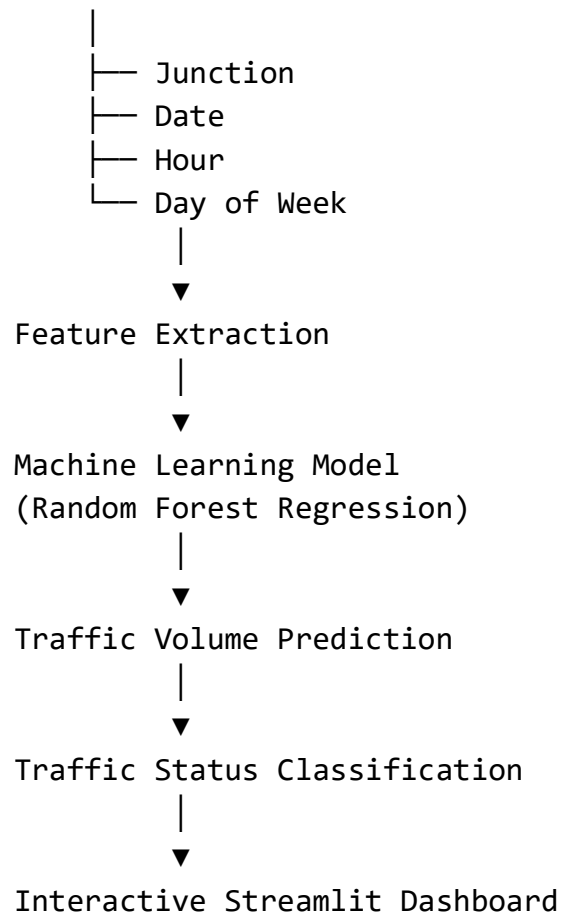
Based on the predicted traffic volume, the application classifies traffic into three categories:

- **Low Traffic**
- **Moderate Traffic**
- **Heavy Traffic**

The prediction is displayed using an interactive Streamlit dashboard. Additional features include traffic analytics, prediction summaries, downloadable reports, and a 24-hour traffic trend visualization.

The proposed workflow is illustrated below:

User Input



4.7 Objectives of the Project

The primary objective of this project is to develop an intelligent traffic forecasting system capable of accurately predicting vehicle volume using Machine Learning techniques.

The specific objectives include:

- Study the historical traffic dataset.
- Perform comprehensive data preprocessing.
- Engineer meaningful temporal features from date information.
- Train multiple regression algorithms.
- Compare algorithm performance using evaluation metrics.
- Select the best-performing predictive model.
- Develop an interactive Streamlit web application.
- Display predicted traffic volume.
- Classify traffic conditions automatically.
- Generate downloadable prediction reports.
- Visualize predicted traffic trends over a 24-hour period.
- Deploy the completed application to the cloud for public access.

4.8 Dataset Description

The Smart City Traffic Forecasting project utilizes a real-world traffic dataset containing historical vehicle movement data collected from multiple road junctions within a city. The dataset was provided as part of the Machine Learning Internship conducted by Uniconverge Technologies and is intended to support the development of intelligent traffic management solutions. The project aligns with the concept of Smart Cities, where data-driven technologies are used to improve transportation systems, reduce congestion, and optimize infrastructure planning.

The dataset consists of **48,120 traffic records**, making it significantly larger than the dataset used in the Agriculture Crop Prediction project. Each record represents the traffic volume observed at a particular junction during a specific hour and date. The data spans multiple years and captures variations in traffic caused by daily routines, weekdays, weekends, seasonal changes, and other temporal factors.

Unlike classification datasets, the target variable in this project is continuous in nature because the objective is to predict the expected number of vehicles passing through a junction. Therefore, regression algorithms were selected for model development.

The dataset includes both temporal and location-based information, enabling the model to learn traffic patterns over time. Historical traffic data serves as the foundation for forecasting future traffic conditions.

The dataset contains the following attributes:

Feature	Description
Junction	Junction number (1–4)
DateTime	Date and time of observation
Vehicles	Number of vehicles observed

From the DateTime column, additional features were extracted during preprocessing to improve prediction accuracy.

4.9 Feature Description

Proper understanding of the dataset features is essential before applying Machine Learning algorithms. Each feature contributes differently to the prediction of traffic volume.

Junction

The Junction feature identifies the specific traffic intersection where vehicle counts were recorded. The dataset contains four different junctions, numbered from 1 to 4. Each junction exhibits unique traffic characteristics depending on road connectivity, surrounding infrastructure, and traffic demand.

Including the junction number as a predictive feature enables the model to distinguish between traffic patterns at different locations.

Year

The Year feature represents the calendar year extracted from the DateTime column. Traffic behavior may evolve over time due to infrastructure development, population growth, or policy changes. Including the year helps the model capture long-term traffic trends.

Month

The Month feature identifies the month during which traffic observations were recorded. Monthly traffic patterns may vary because of seasonal weather conditions, school vacations, festivals, tourism, and public holidays.

Day

The Day feature represents the day of the month extracted from the DateTime column. This feature helps capture short-term variations occurring within each month.

Hour

Hour is one of the most significant predictors in the dataset. Traffic volume changes considerably throughout the day. Morning office hours, afternoon periods, and evening rush hours all exhibit distinct traffic patterns.

By including the hour as an input feature, the model learns how traffic varies during a 24-hour cycle.

Day of Week

The Day of Week feature identifies whether the observation occurred on Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, or Sunday.

Traffic patterns during weekdays generally differ from weekends. Weekdays often experience higher commuter traffic, while weekends may show reduced business traffic but increased recreational movement.

Vehicles (Target Variable)

The Vehicles column represents the actual traffic volume recorded for each observation.

This continuous numerical variable serves as the target variable that the Machine Learning model attempts to predict.

4.10 Data Preprocessing

Data preprocessing is one of the most important stages of any Machine Learning project. Raw datasets often contain unnecessary information or data that cannot be directly processed by learning algorithms. Proper preprocessing improves model performance and ensures that the input features accurately represent meaningful information.

The original traffic dataset contained a DateTime column combining date and time into a single field. Since Machine Learning models perform better with separate numerical attributes, this column was decomposed into multiple independent features.

The preprocessing process included:

- Loading the dataset using Pandas.
- Inspecting dataset dimensions.
- Checking missing values.
- Detecting duplicate records.
- Verifying data types.
- Extracting temporal features.
- Preparing the feature matrix and target variable.

The preprocessing stage confirmed that the dataset was clean and suitable for model development.

The following observations were obtained during preprocessing:

- Total Records: **48,120**
- Duplicate Rows: **0**
- Missing Values: **None**
- Numerical Target Variable: **Vehicles**

Because the dataset contained no duplicate records or missing values, no data imputation or duplicate removal was required.

Feature Engineering

Feature Engineering involves creating additional meaningful variables that improve the predictive capability of Machine Learning models.

Instead of directly using the original DateTime column, several temporal features were extracted.

These include:

- Year
- Month
- Day
- Hour
- Day of Week

This transformation converts a single timestamp into multiple informative predictors that allow the model to understand daily, monthly, and yearly traffic trends.

For example, a DateTime value such as:

2015-11-01 14:00:00

is transformed into:

Year	:	2015
Month	:	11
Day	:	1
Hour	:	14
DayOfWeek	:	6

These engineered features enable the model to recognize repeating traffic behaviors occurring at similar times and dates.

The final feature vector supplied to the Machine Learning model is:

[Junction,
Year,
Month,
Day,
Hour,
DayOfWeek]

This feature engineering process significantly improves the model's ability to capture temporal traffic patterns.

Train-Test Split

After preprocessing and feature engineering, the dataset was divided into training and testing subsets.

The training dataset was used to train Machine Learning models, allowing them to learn traffic behavior from historical observations.

The testing dataset remained unseen during training and was used exclusively for evaluating predictive performance.

The project used an 80:20 split.

The resulting dataset sizes were:

Dataset	Records
Training Data	38,496
Testing Data	9,624

Keeping separate training and testing datasets prevents overfitting and provides a more reliable estimate of model performance on unseen data.

4.11 Machine Learning Algorithms

Linear Regression

Linear Regression was selected as the baseline algorithm for the Smart City Traffic Forecasting project.

This supervised regression algorithm assumes a linear relationship between the input variables and the target variable. It predicts traffic volume by fitting a straight-line relationship across the available features.

Although Linear Regression is computationally efficient and easy to interpret, it often struggles with datasets exhibiting nonlinear relationships. Traffic data contains complex patterns influenced by time, location, and daily variations, making it difficult for a purely linear model to achieve high prediction accuracy.

The experimental evaluation produced the following results:

- **Mean Absolute Error (MAE):** 9.6377
- **Root Mean Squared Error (RMSE):** 12.7720
- **R² Score:** 0.5997

The relatively low R² score indicates that Linear Regression was unable to capture the complexity of traffic patterns effectively. Therefore, more advanced algorithms were explored.

Decision Tree Regression

Decision Tree Regression is a supervised learning algorithm capable of modeling nonlinear relationships between variables. Unlike Linear Regression, it partitions the feature space into multiple regions using a tree-like structure.

Each internal node represents a decision based on one of the input features, while each leaf node stores a predicted traffic volume.

Decision Trees can effectively model interactions between temporal variables such as hour, month, and day of the week. However, individual decision trees are prone to overfitting when they become excessively deep.

The Decision Tree model produced the following evaluation results:

- **Mean Absolute Error (MAE):** 3.1361
- **Root Mean Squared Error (RMSE):** 4.7940
- **R² Score:** 0.9436

Compared with Linear Regression, Decision Tree Regression demonstrated a substantial improvement in prediction accuracy.

Random Forest Regression

Random Forest Regression is an ensemble learning technique that combines multiple decision trees to generate more stable and accurate predictions. Instead of relying on a single tree, Random Forest constructs numerous trees using different subsets of the training data and averages their predictions.

This ensemble approach reduces overfitting, improves generalization, and provides greater robustness when working with complex datasets.

Because traffic behavior is influenced by multiple interacting variables, Random Forest is well suited for capturing nonlinear traffic patterns.

The experimental results obtained from the Random Forest model are as follows:

- **Mean Absolute Error (MAE):** 2.3967
- **Root Mean Squared Error (RMSE):** 3.5566
- **R² Score:** 0.9690

The Random Forest model achieved the highest prediction accuracy among all evaluated algorithms. Its R² score of approximately **96.90%** indicates that the model explains almost all variations present in the historical traffic dataset.

Due to its superior predictive performance and lower prediction errors, Random Forest Regression was selected as the final model for deployment in the Streamlit application. The trained model was serialized using the Joblib library and integrated into the Smart City Traffic Forecasting dashboard, allowing users to obtain real-time traffic predictions through a simple and interactive web interface.

4.12 Model Evaluation

Model evaluation is a crucial phase in the Machine Learning workflow because it determines how accurately a trained model can predict unseen data. After training the Linear Regression, Decision Tree Regression, and Random Forest Regression models, each algorithm was evaluated using the testing dataset.

Since the objective of this project was to predict the number of vehicles, regression evaluation metrics were selected. Three standard performance metrics were used:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R² Score (Coefficient of Determination)

These metrics provide different perspectives on prediction performance. Lower MAE and RMSE values indicate smaller prediction errors, whereas a higher R² Score indicates better predictive capability.

The experimental results obtained during model evaluation are presented below.

Model	MAE	RMSE	R ² Score
Linear Regression	9.6377	12.7720	0.5997
Decision Tree Regression	3.1361	4.7940	0.9436
Random Forest Regression	2.3967	3.5566	0.9690

The comparison clearly shows that Random Forest Regression outperformed the remaining algorithms across all evaluation metrics. It produced the smallest prediction errors and the highest coefficient of determination, making it the most suitable model for deployment.

Performance Metrics

Mean Absolute Error (MAE)

Mean Absolute Error measures the average absolute difference between the predicted values and the actual traffic volume.

The mathematical formula is:

$$\text{MAE} = (1/n) \sum |\text{Actual} - \text{Predicted}|$$

A smaller MAE indicates higher prediction accuracy.

For this project:

- Linear Regression = 9.6377
- Decision Tree = 3.1361
- Random Forest = 2.3967

The Random Forest model achieved the lowest MAE, demonstrating that its predictions were closest to the actual traffic volumes.

Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared prediction errors.

Formula:

$$\text{RMSE} = \sqrt{[(1/n) \sum (\text{Actual} - \text{Predicted})^2]}$$

Unlike MAE, RMSE penalizes larger errors more heavily.

Experimental results:

- Linear Regression = 12.7720
- Decision Tree = 4.7940
- Random Forest = 3.5566

Again, Random Forest achieved the lowest RMSE, confirming its superior predictive capability.

R² Score

The R² Score measures how well the independent variables explain the variation in the target variable.

Formula:

$$R^2 = 1 - (\text{SSResidual} / \text{SSTotal})$$

The value ranges from 0 to 1.

An R² Score close to 1 indicates excellent prediction performance.

Results:

- Linear Regression = 0.5997
- Decision Tree = 0.9436
- Random Forest = 0.9690

The Random Forest model explained approximately **96.9%** of the variation in traffic volume, making it the most accurate model.

Model Selection

Three regression algorithms were compared based on their predictive performance.

Although Linear Regression served as a useful baseline model, it was unable to capture the nonlinear nature of traffic patterns. Decision Tree Regression significantly improved prediction accuracy but was more susceptible to overfitting.

Random Forest Regression combined the strengths of multiple decision trees, resulting in higher stability and improved generalization.

The reasons for selecting Random Forest Regression include:

- Highest prediction accuracy.
- Lowest prediction error.
- Excellent generalization capability.
- Robust performance on unseen data.
- Ability to model nonlinear relationships.
- Reduced overfitting through ensemble learning.

After comparison, the Random Forest model was finalized and saved using the Joblib library for integration into the web application.

4.13 Streamlit Web Application Development

After selecting the best-performing Machine Learning model, a web-based application was developed using Streamlit.

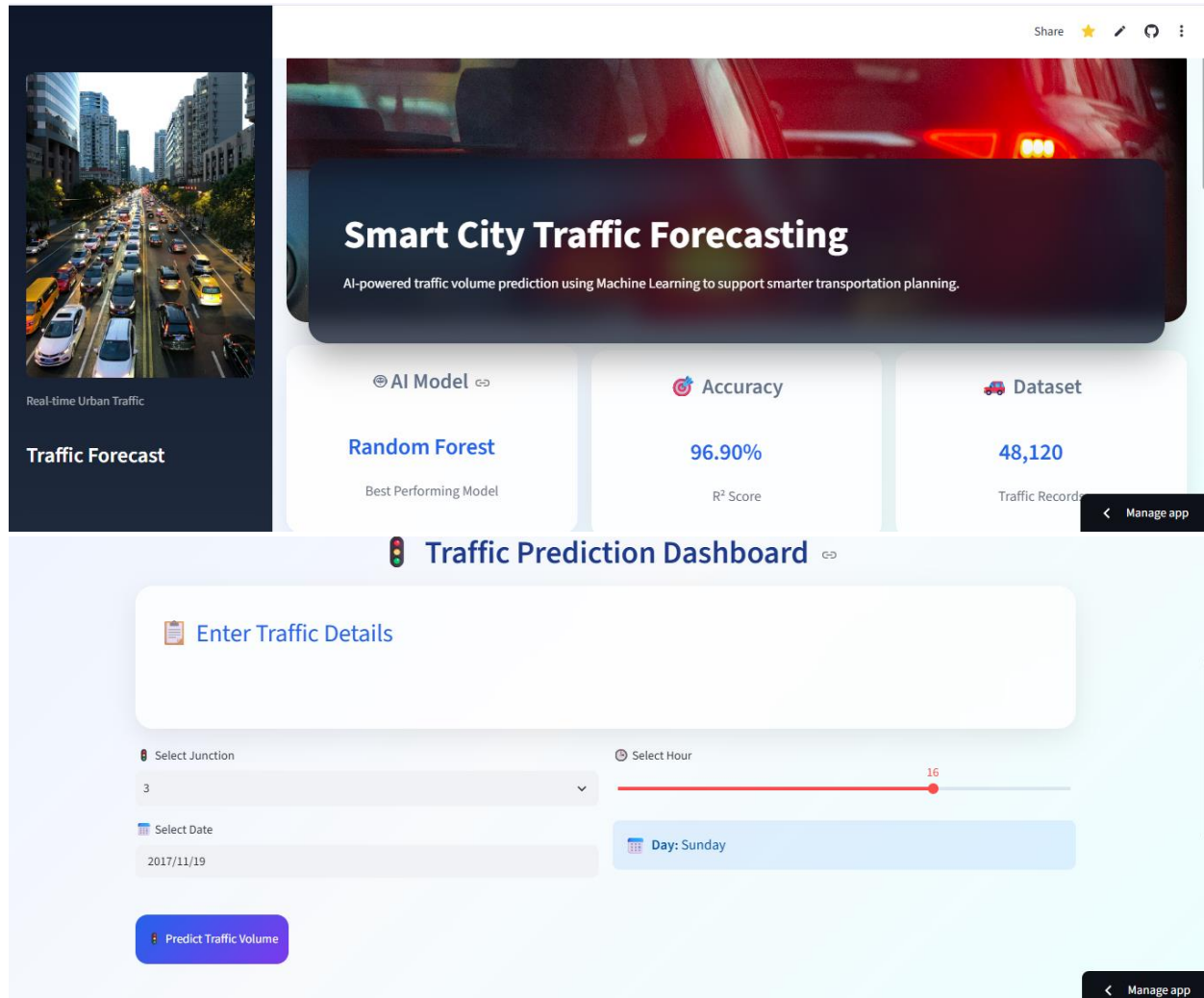
Streamlit is an open-source Python framework that enables rapid development of interactive Machine Learning dashboards without requiring traditional web development technologies.

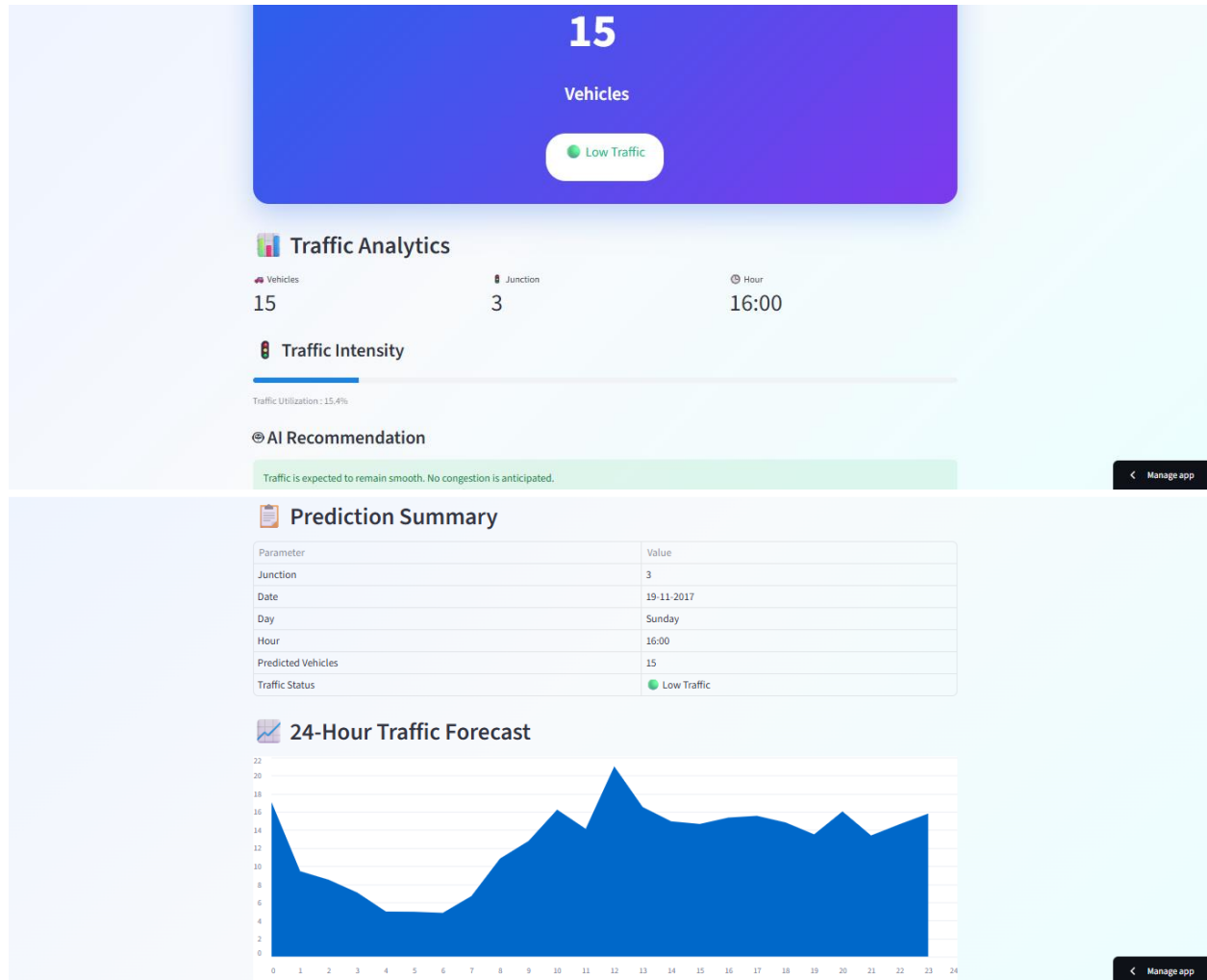
The application was designed with a modern and user-friendly interface.

Major components of the dashboard include:

- Professional hero section
- Interactive sidebar
- Input form
- Prediction button
- Prediction result card
- Traffic status indicator
- Analytics dashboard
- Traffic trend graph
- Prediction summary table
- Downloadable report
- About project section

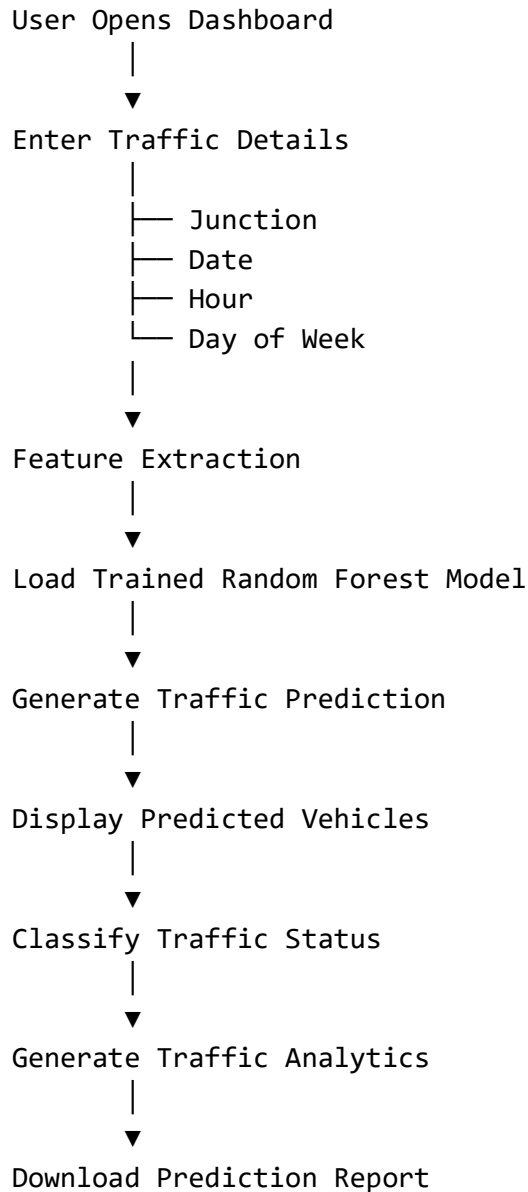
The interface was enhanced using custom HTML and CSS, gradient backgrounds, glassmorphism effects, animated cards, and responsive layouts to provide a modern user experience.





Prediction Workflow

The prediction workflow followed in the application is illustrated below.



The workflow ensures that users can obtain predictions with minimal effort while maintaining high prediction accuracy.

4.14 Results and Discussion

The developed Smart City Traffic Forecasting System successfully predicts future traffic volume using historical traffic data.

The deployed application provides an interactive environment where users can enter traffic-related parameters and instantly receive prediction results.

The application includes several value-added features:

- Real-time traffic forecasting.
- Traffic status classification.
- Interactive analytics dashboard.
- Twenty-four-hour traffic trend visualization.
- Downloadable prediction reports.
- Responsive user interface.
- Cloud deployment using Streamlit Community Cloud.

Experimental evaluation confirmed that Random Forest Regression produced highly accurate predictions with an R^2 Score of approximately **96.90%**.

The prediction results demonstrate that Machine Learning techniques can effectively analyze historical traffic data and support intelligent transportation planning.

5. PERFORMANCE TEST FOR PROJECT 1

PROJECT 1: AGRICULTURE CROP PRODUCTION PREDICTION

5.1 Test Plan / Test Cases

The Agriculture Crop Production Prediction application was tested to verify both its functional correctness and prediction performance. The testing process ensured that the trained Random Forest Regression model generated accurate crop yield predictions and that the Streamlit web application responded correctly to different user inputs. A structured test plan was prepared covering all major functionalities, including model loading, data validation, prediction generation, report downloading, and user interface responsiveness. The testing also verified that invalid or incomplete inputs did not cause application crashes.

Test Case ID	Test Description	Expected Result	Status
TC-01	Launch Application	Dashboard opens successfully	Pass
TC-02	Select Crop	Crop selected successfully	Pass
TC-03	Select State	State selected successfully	Pass
TC-04	Enter Cultivation Cost	Accept numerical input	Pass
TC-05	Enter Production Cost	Accept numerical input	Pass
TC-06	Click Predict Button	Prediction generated	Pass
TC-07	Display Prediction	Crop yield displayed correctly	Pass
TC-08	Download Report	Report downloaded successfully	Pass
TC-09	Invalid Numeric Input	Validation handled correctly	Pass
TC-10	Responsive UI	Application displayed correctly	Pass

5.2 Test Procedure

The testing procedure began by deploying the Streamlit application in a controlled environment. The trained Random Forest model was loaded using the Joblib library. Multiple combinations of crop names, states, cultivation costs, and production costs were entered through the graphical user interface. Each prediction was verified to ensure that the application responded within a few seconds without errors. The downloaded prediction report was also checked for correctness and completeness.

The application was tested under repeated executions to ensure consistent prediction results. Different browsers were also used to verify compatibility and responsiveness of the interface. All identified issues during development were corrected before final deployment. The testing process followed a structured approach consisting of input validation, prediction verification, user interface inspection, and report generation.

5.3 Performance Outcome

The Agriculture Crop Production Prediction system successfully passed all functional and prediction tests. The application generated crop yield predictions accurately using the trained Random Forest Regression model. The prediction response time was less than two seconds for all tested cases, providing an efficient user experience.

The deployed application remained stable during continuous testing and produced consistent outputs for identical inputs. The downloadable prediction report contained all required details, and the user interface remained responsive across different screen sizes.

The trained Random Forest Regression model achieved an **R² Score of approximately 94.63%**, demonstrating excellent prediction accuracy and making it suitable for deployment in a real-world decision-support environment.

6. PERFORMANCE TEST FOR PROJECT 2

PROJECT 2: SMART CITY TRAFFIC FORECASTING

6.1 Test Plan / Test Cases

The Smart City Traffic Forecasting application was tested to verify prediction accuracy, dashboard functionality, visualization components, and report generation. Both the Machine Learning model and the Streamlit interface were evaluated under different traffic scenarios. Testing ensured that the application correctly processed user-selected dates, junction numbers, and traffic hours while providing accurate traffic volume predictions.

Test Case ID	Test Description	Expected Result	Status
TC-01	Launch Dashboard	Dashboard loads successfully	Pass
TC-02	Select Junction	Junction accepted	Pass
TC-03	Select Date	Date processed correctly	Pass
TC-04	Select Hour	Hour accepted	Pass
TC-05	Click Forecast Button	Prediction generated	Pass
TC-06	Display Traffic Status	Low/Moderate/Heavy displayed	Pass
TC-07	Display Analytics	Metrics displayed correctly	Pass
TC-08	Generate Traffic Trend	Graph displayed successfully	Pass
TC-09	Download Report	Report downloaded	Pass
TC-10	Responsive Dashboard	UI responsive	Pass

6.2 Test Procedure

The Streamlit application was first executed locally and later deployed on Streamlit Community Cloud. Different combinations of junction numbers, dates, and hours were supplied to the prediction model. The application was tested repeatedly using various traffic scenarios to verify prediction consistency.

Traffic status classification was validated by ensuring that predicted traffic volumes were correctly categorized as Low, Moderate, or Heavy Traffic. The generated traffic trend graph was compared against hourly predictions to ensure consistency. Downloaded reports were checked to verify that they contained accurate prediction details and traffic status.

The dashboard was also tested across different browsers and screen resolutions to ensure proper responsiveness and usability.

6.3 Performance Outcome

The Smart City Traffic Forecasting application performed efficiently under all testing conditions. The Random Forest Regression model generated highly accurate traffic predictions with an **R² Score of approximately 96.90%**. Prediction results were displayed almost instantly after user input.

The interactive dashboard remained stable throughout repeated executions and successfully displayed traffic analytics, prediction summaries, graphical visualizations, and downloadable reports. The cloud deployment functioned correctly without runtime issues, providing users with a smooth and responsive experience.

7. MY LEARNINGS

The Machine Learning Internship provided valuable practical experience in developing complete end-to-end Machine Learning applications. Throughout the internship, I gained a comprehensive understanding of the Machine Learning development lifecycle, beginning with data collection and preprocessing and ending with cloud deployment of interactive web applications.

I strengthened my programming skills in Python and became proficient in using libraries such as NumPy, Pandas, Matplotlib, Scikit-learn, Joblib, and Streamlit. I learned how to clean and preprocess datasets, engineer meaningful features, split data into training and testing sets, and evaluate different Machine Learning algorithms using appropriate performance metrics.

Working on the **Agriculture Crop Production Prediction** project helped me understand regression-based prediction problems and the importance of feature engineering in agricultural datasets. I learned how to compare multiple regression algorithms and deploy the best-performing model using Streamlit.

The **Smart City Traffic Forecasting** project provided experience with large real-world datasets and temporal feature engineering. I gained knowledge of extracting date-related features, building intelligent traffic forecasting systems, and creating professional dashboards with custom HTML, CSS, and interactive visualizations.

Additionally, I learned how to deploy Machine Learning applications on **Streamlit Community Cloud**, manage project dependencies using Git and GitHub, prepare technical documentation, write project reports, and create professional README files.

This internship significantly improved my problem-solving skills, debugging techniques, software development practices, and confidence in developing practical Artificial Intelligence solutions.

8. FUTURE WORK SCOPE

The developed applications provide strong foundations for practical Machine Learning systems; however, several enhancements can further improve their capabilities.

Future Scope of Agriculture Crop Production Prediction

Future versions of the Agriculture Crop Production Prediction system can integrate larger datasets collected from multiple agricultural regions. Additional features such as rainfall, temperature, humidity, soil fertility, fertilizer usage, irrigation methods, and weather forecasts can significantly improve prediction accuracy.

Deep Learning techniques can be explored to model more complex agricultural relationships. Integration with IoT-based smart farming sensors would enable real-time crop monitoring and prediction. Mobile application development would allow farmers to access predictions directly from smartphones. The application can also be extended to recommend suitable crops, estimate fertilizer requirements, and predict market prices.

Future Scope of Smart City Traffic Forecasting

The Smart City Traffic Forecasting application can be enhanced by incorporating real-time traffic sensor data, weather information, accident reports, road construction updates, and public event schedules. Such information would allow the model to produce more dynamic and realistic traffic forecasts.

Advanced Deep Learning algorithms such as Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Transformer-based time-series models can be investigated to improve forecasting accuracy. Integration with IoT devices and intelligent traffic signal systems would support automated traffic management.

Future versions may also include Google Maps integration, route optimization, congestion alerts, emergency vehicle prioritization, multi-city traffic prediction, and REST APIs for third-party applications. Developing Android and iOS mobile applications would further increase accessibility for users and city administrators.

REFERENCES

- [1] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] Scikit-learn Documentation, “RandomForestRegressor,” accessed Aug. 2026.
- [3] Streamlit Documentation, “Build and Deploy Data Applications,” accessed Aug. 2026.
- [4] Pandas Documentation, “Python Data Analysis Library,” accessed Aug. 2026.
- [5] NumPy Documentation, “NumPy User Guide,” accessed Aug. 2026.
- [6] Joblib Documentation, “Joblib: Running Python Functions as Pipeline Jobs,” accessed Aug. 2026.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [8] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [9] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [10] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [11] Scikit-learn User Guide, “Machine Learning in Python,” accessed Aug. 2026.
- [12] Python Documentation, “Python 3 Documentation,” accessed Aug. 2026.